

“ بنام ایزد دانا ”

گزارش کار پروژه معماری کامپیوتر : *CPUlator*

هانیہ تبیانان - 402243048

شرح کلی نحوه پیاده سازی :

پروژه ما طوری طراحی شده که نیازمند 3 حافظه برای اشخاص و 1 حافظه برای تعداد اشخاص ورودی آمده است میباشد. حافظه اول بنام $data - name$ این حافظه اسم اشخاص را در خود نگه میدارد، حافظه بعدی $money$ است که نظیر به نظیر حافظه اسم ها در آن پول (دارایی) فرد را نگه میدارد. (وقتی شخص جدیدی وارد سامانه میشود دارایی آن 0 است) . حافظه بعدی سعی در پیاده سازی ماتریس دارد، این برای این است که بتوانیم رابطه قرض دهی هر فرد به فرد دیگر را پیاده سازی کنیم.

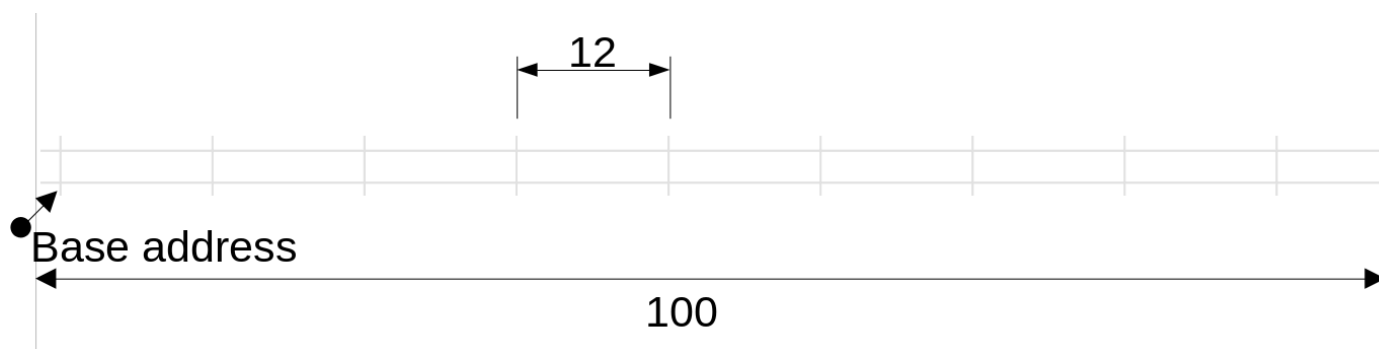
این پروژه دارای 6 نوع دستور بوده است . این شش دستور دارای یه ویژگی یکسانند و آن هم این است که شماره هر دستور ابتدای آن دستور آمده است همین سبب میشود که وقتی ما رشته دستور را دریافت میکنیم بنا بر اولین کاراکتر آن که عددی از 1 تا 6 است به آن دستور مربوطه میرویم .

۱. دستور اول :

این دستور به فرمت $1\ s_1\ s_2\ x$ است. در ابتدا باید هر یک از رشته ها را در صورت وجود بیابیم یا هر صورت عدم وجود آنرا بوجود بیاوریم .

از یه لیبل استفاده میکنیم که برای ما حکم تابع را دارد . این تابع به ما در یافتن $index$ آن رشته کمک میکند ، این تابع اینگونه کار میکند که در حافظه ای که اسم اشخاص سیو شده است میگردد تا آن رشته اسم را پیدا کند اگر نتوانست آنرا پیدا کند در خانه ای خالی از این حافظه (دقیقاً خانه خالی بعد از آخرین اسمی که نوشته شده است) و بعد $index$ آنرا سیو میکند.

A. تابع $find - index - name$: این تابع از نظر من قلب طراحی من است . در حافظه $data - name$ میچرخد تا اسم رشته ای که آمده است را پیدا کند اینگونه که در ابتدای به تعداد اسم هایی که آمده اند در حافظه میچرخد تا اسم را پیدا کند و حرف به حرف تطبیق بدهد اگر تطبیق داده شد اولین خانه ای که اسم در آن شروع شده را در جایی ذخیره میکند بعد از آنجا که این حافظه 100×12 اسم یعنی اگر نگاه ماتریسی و ارایه ای به آن داشته باشیم اندازه هر خانه 12 تا است برای اینکه $index$ را در بیاریم باید موقعیت آن را از موقعیت جایی که حافظه شروع میشود کم کنیم و عدد حاصله را بر 12 تقسیم کنیم اینگونه است که عدد $index$ (چندمین اسم است ، با فرض اینکه شماره گذاری از 12 شروع میشه) :



این تابع در دستور های 1 و 4 و 5 و 6 خیلی کارایی دارد و خیلی کمک کننده است. وقتی شماره ایندکس را داشته باشیم میتوانیم به همان ایندکس از حافظه پول هم دسترسی داشته باشیم که این حافظه پول ، در همان ماتریس اختلاف پول داده شده و گرفته شده را نمایش میدهد ، و این به معنی این است که زمانی که نام جدید نوشته میشود در خانه نظیر آن در حافظه پول 0 نوشته میشود .

یک حافظه دیگر هم خواهیم داشت که این حافظه در سطح بالا برای ما ماتریس را شبیه سازی میکند ، ماتریس هم نحوه ذخیره سازی ان شبیه ارایه تک بعدی است اما با این تفاوت که اگر ماتریس را در نظر بگیریم در حافظه دستگاه از جایی که حافظه ماتریس شروع میشه 100 خانه جلو میرود که این خانه برای اون لایینی از ماتریس است که ایندکس ای از همان اسمی که ما میخواهیم روی ان سرچ انجام بدهیم . در واقع هر اسمی میتواند با اسم دیگر در ارتباط باشد این ماتریس همان ارتباط را بوجود میآورد که اسم های ردیف عمودی (در ماتریس) به اسم های ردیف افقی پول میدهد (اگر بر خلاف این باشد عدد منفیه). فلسفه عدد 100 هم این اسم که حداکثر اسم هایی که میاید 100 تا است .

وقتی ایندکس های دو اسم را داریم باید به همان خانه از ماتریس برسیم که خواهیم داشت که از جایی که حافظه شروع میشود شروع میکنیم به اندازه تعداد ایندکس هایی که پشت ایندکسی هستند که میتوانیم روی ان سرچ کنیم لاین افقی طی میکنیم که 100 تا خانه 4 بایتی میشود و برای رسیدن به خانه مد نظر باید به اندازه ایندکس اسمی که مطلوبمان است خانه 4 بایتی طی کنیم پس در واقع نحوه کارکرد دستور 6 ام مشخص شد .

نحوه کارکرد دستور های 5 و 4 اینگونه است که در لایینی از ماتریس سرچ انجام میدهد که ایندکس اسمی است که بعد از عدد 5 یا 4 آمده است بنا بر اینکه چه مطلوبی که دستور ها میخواهد اینکه عدد نوشته شده در خانه های 4 بایتی منفی است یا مثبت عدد مربوطه را زیاد میکند .

اما دستور 2 و 3 باید در حافظه پول بچرخد و به دنبال کوچکترین و بزرگترین عدد بچرخد که ساده است اما چیز مهم در این دو دستور این است که باید ترتیب لغت نامه هم حفظ شود بخاطر همین در بین اسم هایی که پول یکسان دارند میچرخیم و به دنبال اسمی هستیم که از نظر لغت نامی ای در موقعیت پایین تری است . باید در صورت وجود حرف به حرف بخواند و کد اسکی ان دو را از هم کم کند و ببیند مثبت بود یا منفی تغییر بدهد یا ندهد .

برای اینکه عددی که دستور اول به صورت رشته آمده است را به عدد تبدیل کنیم از تکنیک اینکه اگه رشته کاراکتر 1 بود تو به علاوه 1 کن و این دست استفاده میکنیم و عدد را بصورت $100 \times$ نگهداری میکنیم که بتوانیم زمانی که میخواهیم انرا بصورت ممیزی چاپ کنیم با دو رقم اعشار بر 10 دو بار تقسیم کنیم و مود بگیریم

در طی پروژه از این تکنیک استفاده شده که بعضی کارها بین دستور ها مشترک هست پس باید به شناسه داشته باشیم که عدد دستور را در خودش نگه میدارد و بنا بر اینکه این شناسه چه عددی است به ان دستور بپرد .

شرح با جزئیات :

در ابتدا که شروع میشود ما تعداد دستور ها را میگیریم و تعداد اسم های که آمده اند را در حافظه تعداد اسم با عدد 0 مینویسم. در مرحله بعد در حلقه ای میچرخیم و بنا بر اینکه چه دستوری به آن دستور میپریم با گرفتن کاراکتر اول دستور

دستور اول : این دستور با فرم " $s_1 s_2 x$ " با خواندن کاراکتر اول باید 2 تا جلو برویم چون بینشان فاصله است تا به کاراکتر اول اسم نخست برسیم در این بخش به دنبال ایندکس این اسم در تابع میگردیم و در صورت عدم وجود آنرا ایجاد کرده و پول آنرا 0 قرار میدهیم (هر چند که خودش در ابتدا صفر مقدار دهی میشود) بعد از آن که ایندکس آنرا یافتیم آنرا در s_6 نگهداری میکنیم و با ادامه رشته دستور کارمان را سر میگیریم و همین کار ها را برای اسم دوم هم میکنیم و ایندکس آنرا در s_7 نگهداری میکنیم در مرحله بعد نیاز است که عدد داده شده که بصورت رشته است را اول به عدد تبدیل کنیم و چون عدد بصورت اعشاری و با دو عدد بعد از ممیز داده میشود ما عدد مربوطه را 100 برابر کرده و نگهداری میکنیم و در صورت نیاز و چاپ آنرا دو بار مکرر بر 10 تقسیم کرده و در مکان های ممیز قرار میدهیم و این عدد را از پولی که این اسم ها در ایندکس های نظیر در حافظه پول داشتند از اسم اول کم میکنیم و به اسم دوم اضافه میکنیم در مرحله بعد باید این مقدار را هم در ماتریس هم ذخیره کنیم به 2 خانه مربوط یکبار در لاین ایندکس اسم اول و در مکان ایندکس اسم دوم عدد را همانطور که هست قرار میدهیم و یکبار هم در لاین اسم دوم و مکان ایندکس اسم اول عدد پول را در حالت منفی قرار میدهیم در واقع بیشترین حجم کاری در همین دستور اول رخ میدهد .

دستور دوم : این دستور با فرم "2" میخواهد اسمی را که بنا بر حروف الفبای انگلیسی کوچکتر است و بیشترین پول را دارد چاپ کنیم . در واقع این دستور با حافظه پول در ابتدا سروکار دارد در این حافظه میچرخد و اون عددی را که بیشینه است را پیدا میکند در مرحله بعد باید باز در این حافظه بچرخد و ببیند آیا اسمی هست که همین مقدار پول را داشته باشد اگر وجود داشت بنا بر اینکه اسم کوچکتر بود یا نه باید ایندکس را تغییر بدهد .

دستور سوم : این دستور با فرم "3" همانند دستور دوم کار میکند با این تفاوت که به دنبال اسمی با کمترین پول است پس باید پی کمترین پول بود چیزی که در دستور دوم و سوم مهم است این است که این دستورها اگر اسمی وجود نداشت (چون اکیدا باید منفی یا مثبت باشد) باید عدد منفی یک را چاپ کنیم که انهم در دستور ها لحاظ و اجرا شده

دستور چهارم : این دستور با فرم " $s_1 4$ " به دنبال تعداد افرادی میگردد که اسم s_1 باید به آنها پول بدهد در واقع ما با یافتن ایندکس این رشته اسم باید در لاین آن در ماتریس بچرخیم و عدد ها را بخوانیم و جلو برویم اگر عدد منفی بود یعنی باید تعداد این افراد را افزایش بدهیم

دستور پنجم : این دستور با فرم " $s_1 5$ " شبیه دستور 4 پیاده سازی شده فقط با این تفاوت که اگر عدد خوانده شده مثبت بود باید افزایش دهیم

دستور ششم : این دستور با فرم " $s_1 s_2 6$ " به لطف نحوه نگهداری و پیاده سازی خیلی راحت است باید فقط دو تا ایندکس ها از دو اسم را در بیاوریم و به همان خانه رجوع کنیم فقط با این تفاوت در کد پیاده سازی شده که بر خلاف دستور اول که اسم نخست

به اسم ثانی پول میدهد در این دستور باید ببینیم که اسم نخست چقدر باید به اسم دوم بدهد تا تصویه شود ما به خانه تقارنی
ان میرویم و اون عدد را چاپ میکنیم چون عدد منفی یک برابر ان است و راحت تر است .
برای چاپ هم به صورت عدد اعشاری دو بار مکرر بر عدد 10 مود میگیریم و تقسیم میکنیم و در 3 تا رجیستر نگهداری میکنیم و
اول عدد ماقبل اعشار را چاپ میکنیم و بعد خود ممیز را چاپ میکنیم و دو عدد دیگر را .